

The CMED Integration Specification

Everything CMED builds, in one document — the two browser signals and the two server messages · AIMScribe v3 · AIMS LAB, United International University · 5 October 2026

The whole integration in one paragraph

AIMScribe runs as a small program on **the same laptop** as the doctor's browser. CMED's page connects to it over the computer's own internal network — no internet involved — and tells it when a patient is opened and when the prescription is built, so the recorder knows where one consultation ends and the next begins. Separately, **CMED's server** sends us the patient's information and the prescription, so the recording has a clinical record attached to it. **Audio never goes to CMED, and nothing clinical ever comes back to CMED.**

Every rule in this document is taken from the program that actually handles or checks these messages when they arrive. **If this document and the software ever disagree, the software is right and this is a bug** — please tell us.

What is in here

Part	Covers	Who builds it	Sections
A	How it all fits together, and the five fields that tie it together	read this first	1–2
B	The two browser signals — API 1 and API 3 on Channel A	CMED's front-end developer	3–9
C	The two server messages — API 2 and API 3 on Channel B	CMED's back-end developer	10–15
D	Where the data lands, how to test, what each side hands over	both, and the project leads	16–21

A front-end developer needs Parts A, B and D. A back-end developer needs Parts A, C and D. Nobody needs to read all of it to start.

Three things that cause almost every integration failure

- 1. The five fields must be identical in all four messages** (section 2). Above all, `start_time` in the prescription messages is the time the **patient was opened**, not the time the prescription was built.
- 2. Building the prescription does not stop the recording** (section 7). The counselling afterwards is valuable audio. The consultation closes when the **next** patient is opened.
- 3. CMED must give AIMS LAB two things** (section 20): the exact web address of your page, and your own clinic codes. Without the second one a recording is **silently erased after 24 hours** while CMED sees nothing wrong.

How it all fits together

The four messages, and the five fields that tie them to one consultation

1 · Three APIs — and why they are four messages

	What	From where	When	Carries	Part
API 1	Start recording	the browser	the doctor opens a patient	the five fields	B
API 2	Patient information	CMED's server	the same moment	demographics, paramedic readings, previous visit	C
API 3 Channel A	Prescription built	the browser	the doctor presses Build Prescription	the patient and session id	B
API 3 Channel B	The prescription	CMED's server	the same moment	medicines, diagnoses, investigations	C

On the naming — there are three APIs, not four

CMED's existing integration guide numbers these **API 1**, **API 2** and **API 3**, and that numbering is correct and unchanged. **API 3 is one event** — the prescription has been built — but it has to be told to **two different places**: the recorder on the doctor's PC, so it knows the consultation has reached its end, and the AIMS LAB server, so we receive the prescription itself.

This document therefore writes "**API 3, Channel A**" for the signal to the PC and "**API 3, Channel B**" for the message to our server. They are the two halves of API 3, matching sections 6a and 6b of the earlier guide. **An earlier draft of this document called them API 3a and API 3b; those labels are withdrawn.** Three APIs, four messages.

They go in pairs. When the doctor opens a patient, the browser sends API 1 and your server sends API 2 at the same moment. When the doctor presses Build Prescription, the browser sends API 3 on Channel A and your server sends API 3 on Channel B. Four messages, two moments.

Why two channels instead of one?

The browser signal has to be **instant**, because the microphone must start before the doctor says anything — so it goes straight to the program on the same laptop, with no internet involved. The patient data has to be **trustworthy**, so it travels server to server with a secret key, where no browser and no user can touch or alter it. Neither path could do both jobs well, which is why there are two.

2 · The five fields — the most important rule in this document

Every one of the four messages about a single consultation carries these same five fields, with **exactly the same values, character for character**. They are how we match the prescription to the right recording.

```
{
  "patient_id": "P0012345",
  "doctor_id": "DR0042",
  "hospital_id": "AALO_DHOLPUR",
  "start_time": "2026-10-05T10:14:32+06:00",
  "date": "2026-10-05"
}
```

Field	Type	Rule	Example
patient_id	text	Letters, digits, hyphen, underscore. 1–64 characters. No spaces	P0012345
doctor_id	text	same rule	DR0042

hospital_id	text	same rule. Must be the clinic code AIMS LAB gives you	AALO_DHOLPUR
start_time	text	Date and time with the time-zone offset	2026-10-05T10:14:32+06:00
date	text	The clinic's local date	2026-10-05

Three mistakes that break the match

1. Sending the prescription's own time as start_time. API 3 on Channel A and API 3 on Channel B must repeat the **original** start_time from when the patient was opened — not the time the prescription was built. This is the single most common integration mistake, and it is silent: both messages succeed, and the prescription simply never finds its recording.

2. Leaving off the time-zone offset. 2026-10-05T10:14:32 is rejected. +06:00 must be there.

3. Spaces or other characters in an identifier. These become folder names on disk, so only letters, digits, hyphen and underscore are allowed.

The practical advice: build the five fields once, into a single object, at the moment the doctor opens the patient. Keep that object for the whole consultation and pass the same one to all four messages. Never rebuild them, and never recompute the time. The worked example in section 18 does exactly this.

The two browser signals

API 1 and API 3 on Channel A · built by CMED's front-end developer

3 · How the page connects

ws://127.0.0.1:5050/ws

	Detail
Address	ws://127.0.0.1:5050/ws — the same on every clinic PC. Never a name, never another machine
Protocol	A WebSocket. Messages are JSON, UTF-8
Largest message	64 KB
Password	None. See below — this surprises people
When to connect	When the doctor's page loads. Keep it open; reconnect if it drops

3.1 Why there is no password, and what replaces it

A password in a web page is not a secret — anyone can read it in the page source. So the recorder does not use one. It checks three things instead, all of which a browser sets itself and a page cannot forge:

	What is checked	What it stops
1	The exact web address your page is served from (its "origin"), against a list AIMS LAB holds	Any other website on the internet from driving the recorder
2	The address the connection was made to	A trick where a hostile site makes your own browser connect for it
3	The connection comes from this same computer	Any other machine on the network, full stop

So the one thing CMED must give AIMS LAB for this to work is the exact web address of your page — for example `https://ehr.aaloclinic.com`. Every address you will use, including staging and test sites. An address not on the list is refused with `ORIGIN_NOT_ALLOWED` and the connection never opens.

It must match exactly. `https://ehr.aaloclinic.com` and `https://www.ehr.aaloclinic.com` are two different addresses. So are `http://` and `https://`, and so is a different port number.

4 · The shape of every message and every reply

What CMED sends

```
{
  "command":    "start",           <-- which signal
  "request_id": "abc-123",         <-- anything you like, up to 128 characters
  ...           <-- the fields for that command
}
```

What comes back — always exactly one reply per message

```
{
  "event":      "ack",             <-- "ack" if it worked, "error" if not
  "command":    "start",           <-- the command you sent
  "status":     200,               <-- like an HTTP status
  "code":       "RECORDING_STARTED", <-- ACT ON THIS
  "message":    "Recording started.", <-- wording for a human; may change
  "request_id": "abc-123",         <-- your id, echoed back
  "data":       { ... }           <-- extra detail, per command
}
```

Act on code, **never on** message. The wording is for showing a person and may be reworded at any time; the code is a promise and will not change. Use `request_id` to match a reply to the message that caused it.

5 · API 1 — a patient has been opened

Send this the instant the doctor opens a patient. **The microphone starts immediately**, before any permission check finishes, so nothing of the consultation is missed.

```
{
  "command": "start",
  "request_id": "open-P0012345-1",
  "trigger": {
    "patient_id": "P0012345",
    "doctor_id": "DR0042",
    "hospital_id": "AAL0_DH0LPUR",
    "start_time": "2026-10-05T10:14:32+06:00",
    "date": "2026-10-05"
  }
}
```

Field	Required	Rule
command	yes	the word start
request_id	no	any text up to 128 characters, echoed back
trigger	yes	an object holding the five fields
trigger.patient_id	yes	letters, digits, hyphen, underscore. 1–64. No spaces
trigger.doctor_id	yes	same rule
trigger.hospital_id	yes	same rule. Must be the clinic code AIMS LAB gives you
trigger.start_time	yes	date and time with the offset , e.g. +06:00
trigger.date	yes	YYYY-MM-DD, the clinic's local date

These five values must be identical to the ones in API 2, the server-to-server message you send at the same moment.

5.1 What comes back

```
"data": {
  "session_id": "01M3MK4J4XEDG5NDKE4H5B5XA2",    <-- KEEP THIS
  "started_at": "2026-10-05T10:14:32.865Z",
  "armed": false,
  "previous_session_stopped": true,
  "previous_session_id": "01M3MEYY...",
  "authorisation": "granted",
  "confirmation": "confirming"
}
```

Code you may receive	Meaning	What CMED should do
RECORDING_STARTED (200)	Recording, and permission confirmed	Store the session_id. Show the doctor it is recording
RECORDING_PROVISIONAL (202)	Recording. Permission is still being checked with the server	Treat exactly like success. This is normal, not a warning
SESSION_ALREADY_ACTIVE (409)	This same patient is already being recorded	Nothing. You probably sent it twice
GATE_NOT_ARMED (409)	The previous consultation has not had its prescription built yet	Ask the doctor to build the prescription for the previous patient first
AUTHORISATION_FAILED (401)	The server refused this recording, so it was stopped and deleted	Show the doctor it is not recording. Contact AIMS LAB
CLINIC_MISMATCH (401)	hospital_id is not the clinic this PC belongs to	Stop. A configuration problem — contact AIMS LAB

DOCTOR_NOT_AT_CLINIC (404)	That doctor is not registered at this clinic	Check the doctor_id
MISSING_FIELD (400)	A field is missing or malformed	Fix and resend
INVALID_IDENTIFIER (400)	An id has characters that are not allowed	Fix and resend
DEVICE_NOT_ENROLLED (423)	This PC has not been registered with AIMS LAB	Stop. Contact AIMS LAB
ORIGIN_NOT_ALLOWED (403)	Your page's address is not on the allow-list	Send AIMS LAB the address. See section 3.1
AGENT_NOT_READY (503)	The recorder is starting up	Wait a moment and retry

RECORDING_PROVISIONAL is the one people get wrong. It means the microphone is live and audio is being captured — the server simply has not finished confirming permission yet, which normally takes a fraction of a second. Show the doctor the same "recording" state you would for RECORDING_STARTED. Do not show a warning, and do not retry.

But a provisional start can still fail afterwards. If the server then refuses it, you receive AUTHORISATION_FAILED or DEVICE_NOT_ENROLLED **a moment later, and the recording is stopped and deleted.** So do not treat the first reply as final: keep listening, and if one of those arrives, change what the doctor sees from "recording" to "not recording". The status messages in section 8 are the reliable way to show this.

6 · API 3, Channel A — the prescription has been built

Send this the moment **Build Prescription** succeeds, at the same time as your server sends API 3 on Channel B.

```
{
  "command": "prescription_built",
  "request_id": "built-P0012345-1",
  "patient_id": "P0012345",
  "session_id": "01M3MK4J4XEDG5NDKE4H5B5XA2"
}
```

Field	Required	Rule
command	yes	the words prescription_built
request_id	no	any text, echoed back
patient_id	yes	must be the patient currently being recorded
session_id	yes	the one API 1 gave you for this consultation
occurred_at	no	date and time with offset, if you want to record it

Code	Meaning	What CMED should do
GATE_ARMED (200)	Accepted	Nothing
GATE_ALREADY_ARMED (200)	You sent it twice	Nothing. Harmless
PATIENT_MISMATCH (409)	The patient named is not the one being recorded	Check you used the right session
NO_ACTIVE_SESSION (409)	Nothing is being recorded right now	The consultation already ended

7 · The most important idea in this document

Building the prescription does NOT stop the recording.

The doctor builds the prescription, prints it, hands it to the patient — and then **keeps talking**. That counselling, the part where the doctor explains how to take the medicine and what to watch for, is some of the most valuable audio in the whole study.

So prescription_built does not end anything. It **arms a gate**: it tells the recorder "this consultation has reached its end, so the next patient is allowed to close it".

The recording actually stops when the next patient is opened — when your next API 1 arrives, typically twenty to thirty seconds later.

What the doctor does	What CMED sends	What the recorder does
Opens patient A	API 1 (A) + API 2 (A)	Starts recording A
Consults	—	Recording
Presses Build Prescription	API 3 on Channel A (A) + API 3 on Channel B (A)	Keeps recording. Arms the gate
Prints it, hands it over, counsels	—	Still recording — this is the valuable part
Opens patient B	API 1 (B) + API 2 (B)	Closes A , starts recording B

What if the doctor finishes for the day, or goes to lunch? There is a Stop button on the recorder's own small on-screen control, which the doctor presses. **CMED does not need to send anything**, and must not try to: stopping is a clinical decision with a reason attached, and it belongs to the doctor, not to the page.

8 · Messages the recorder sends you, unprompted

As soon as the connection opens, and whenever something changes, the recorder pushes a status message. You do not have to use it — but it is how you show the doctor what is really happening.

```
{
  "event": "status",
  "state": "recording",           <-- idle | recording | paused | closing
  "is_recording": true,
  "is_paused": false,
  "session_id": "01M3MK4J...",
  "patient_ref": "P0012345",
  "armed": false,                <-- has the prescription signal arrived?
  "authorisation": "granted",
  "confirmation": "confirmed",
  "duration_seconds": 182.4,
  "level": 0.42,                 <-- 0.0 to 1.0, how loud the room is
  "connected_clients": 1
}
```

Useful for	Which field	Suggested use
"Is it recording?"	is_recording	A red dot on the page
"Is the microphone working?"	level	A small meter. A dead microphone sits at 0.0 and is otherwise invisible until the recording is played back
"How long so far?"	duration_seconds	A timer
"Has the prescription signal landed?"	armed	A tick beside Build Prescription

The microphone meter is worth the hour it takes to build. A microphone that has been unplugged, muted in Windows, or set to the wrong input records perfect silence, and nothing on screen looks wrong. Without a meter that is discovered when somebody plays the file back — which may be weeks and hundreds of consultations later.

9 · Commands CMED must not send

The connection also accepts stop, pause and resume. **These are not CMED's to use.** They belong to the doctor, through the recorder's own on-screen control, because each one records a clinical reason that becomes part of the permanent record — including "the patient did not consent", which erases the recording everywhere.

The only two commands CMED should ever send are start and prescription_built.

The two server messages

API 2 and API 3 on Channel B · built by CMED's back-end developer

10 · How CMED's server connects

	Address
API 2 — patient information	POST https://<aims-lab-server>/api/v2/clinical/patient-information
API 3, Channel B — prescription	POST https://<aims-lab-server>/api/v2/clinical/prescription
Header on both	X-CMED-Key: <the key AIMS LAB gives you>
Body	JSON, UTF-8, under 1 MB
Success	HTTP 202 — the message is stored before you get this reply

Also on the same server	Address	Key needed?
Is the system running?	GET /health	No — poll it freely
Which system is this?	GET /	No

The hostname is configuration, not a constant. AIMS LAB sends the staging and production hostnames with the key, and they differ. The server runs on **DigitalOcean in Bangalore**, the lowest-latency region available to Dhaka at roughly 45–70 ms. **That latency never affects recording** — the microphone is driven by the program on the clinic PC over loopback, so it starts instantly whether or not our server is reachable.

Operational detail — endpoints, limits, monitoring, the test commands and the go-live checklist — is in the companion "Integration & Test Guide for CMED DevOps". This document stays with the messages themselves. **The key is a server credential. It must never appear in a browser, in page source, or in anything a user can view. Both messages go server to server.**

Never let these messages delay the doctor. Put them on a background queue and let the page carry on. If our server is briefly unreachable, the consultation still happened and the recording still exists — the clinical record just needs to catch up. A message that arrives an hour late is fine.

11 · API 2 — patient information

Sent the moment the doctor opens the patient, at the same time as the browser's API 1. It carries everything known **before** the consultation.

```
{
  "patient_id": "P0012345", "doctor_id": "DR0042",
  "hospital_id": "AAL0_DHOLPUR",
  "start_time": "2026-10-05T10:14:32+06:00", "date": "2026-10-05",

  "demographics": {
    "name": "Rahima Khatun",
    "sex": "female",
    "age_years": 34,
    "date_of_birth": "1992-03-11",
    "phone": "01700000000",
    "address": "Dholpur, Dhaka"
  },

  "paramedic": {
    "recorded_at": "2026-10-05T10:02:10+06:00",
    "weight_kg": 58.4,
    "height_cm": 157,
    "blood_pressure": "120/80",
    "pulse_bpm": 78,
  }
}
```

```

    "temperature_c": 36.8,
    "spo2_percent": 98,
    "notes": "complains of headache"
  },

  "previous_visit": null,

  "female_details": {},
  "male_details": {}
}

```

11.1 demographics — required

Field name	Type	Accepted	If it is wrong	Goes to
name	text	up to 200 characters. full_name also accepted	shortened	patients.full_name
sex	text	exactly female or male	the whole message is quarantined	patients.sex
age_years	number	0 to 130	stored empty	encounter_demographics
date_of_birth	text	YYYY-MM-DD	stored empty	patients.date_of_birth
phone	text	up to 40 characters	shortened	patients.phone
address	text	up to 500 characters	shortened	patients.address

demographics must be present on every visit, first or fiftieth. **sex is the one field that can fail the whole message** — anything other than the two words is reported back as a problem, so send the word, not a code like M or F.

11.2 paramedic — optional, but send it when you have it

Field name	Type	Accepted	Goes to
recorded_at	text	date and time with offset	paramedic_observations
weight_kg	number	0.5 to 499	weight_kg
height_cm	number	20 to 299	height_cm
blood_pressure	text	"120/80" — upper 30–300, lower 10–250	blood_pressure, and split into systolic and diastolic
pulse_bpm	whole number	10 to 300	pulse_bpm
temperature_c	number	25 to 45	temperature_c
spo2_percent	whole number	0 to 100	spo2_percent
notes	text	up to 2000 characters	notes

A reading outside its range is stored as empty — it never rejects the message. One mistyped temperature must not stop a prescription being saved.

Anything else you put inside paramedic is kept. If you measure something we have not listed — respiration rate, blood sugar, anything — just send it. It is stored in an other field alongside the rest. You do not need our permission and you do not need us to change anything first.

12 · previous_visit — the rule, in full

This field must always be present. Send null when there is nothing to send. Leaving the field out entirely is reported as a problem, because we cannot tell "first visit" from "CMED forgot".

The situation	What to send
The patient's first ever visit	"previous_visit": null
A returning patient, and AIMS LAB does not yet hold the previous visit	the full previous visit, as shown below
A returning patient, and AIMS LAB already holds the previous visit	"previous_visit": null

```
"previous_visit": {  
  "date": "2026-06-02",  
  "prescription": {  
    "items": [  
      { "drug": "Metformin 500 mg", "dose": "1 tablet",  
        "frequency": "1+0+1", "duration": "30 days",  
        "instructions": "after food" }  
    ],  
    "diagnoses": ["Type 2 diabetes mellitus"],  
    "notes": "review in one month"  
  }  
}
```

Field	Rule
date	Required if you send previous_visit at all. YYYY-MM-DD. The date of that earlier consultation. Without it the whole previous visit is ignored
prescription	Same shape as API 3 on Channel B, section 13. Only items really matters
diagnoses, notes	May sit inside prescription or beside it — both work

Sending it more often than necessary is completely safe. If you are ever unsure whether we have the previous visit, send it. The database will not duplicate anything: a repeated previous visit updates the one row we already have. **The dangerous direction is the other one** — deciding not to send it when we do not actually have it, which leaves a permanent hole in that patient's history that nobody notices.

Recommendation for the first release: send it every time for every returning patient. It costs about 2 kilobytes. Once we build an endpoint that lets CMED ask us what we already hold, you can switch to sending it only when needed.

13 · API 3, Channel B — the prescription

Sent the moment the doctor presses **Build Prescription**, at the same time as the browser's API 3 on Channel A. Remember that this does **not** stop the recording — see section 7.

```
{
  "patient_id": "P0012345", "doctor_id": "DR0042",
  "hospital_id": "AAL0_DHOLPUR",
  "start_time": "2026-10-05T10:14:32+06:00",    <-- the ORIGINAL start time
  "date": "2026-10-05",

  "issued_at": "2026-10-05T10:26:55+06:00",
  "items": [
    { "drug": "Napa 500 mg", "dose": "1 tablet",
      "frequency": "1+1+1", "duration": "5 days",
      "instructions": "after food" },
    { "drug": "Omeprazole 20 mg", "dose": "1 capsule",
      "frequency": "1+0+0", "duration": "14 days",
      "instructions": "before breakfast" }
  ],
  "diagnoses": ["Acute gastritis"],
  "investigations": ["CBC", "HbA1c"],
  "advice": "avoid spicy food",
  "follow_up": "2026-11-05",
  "notes": "patient counselled about diet"
}
```

Field name	Type	Required ?	Rule	Goes to
issued_at	text	YES	date and time with offset	prescriptions.issued_at
items	list	YES	one entry per medicine. Send [] if none	prescription_items
items[].drug	text	YES	up to 300 characters. Every item needs one	drug
items[].dose	text	no	up to 200	dose
items[].frequency	text	no	up to 200, e.g. 1+1+1	frequency
items[].duration	text	no	up to 200, e.g. 5 days	duration
items[].instructions	text	no	up to 1000	instructions
diagnoses	list of text	YES	Send [] if none	diagnoses
investigations	list of text	YES	Send [] if none	investigations
advice	text	no	up to 2000	prescriptions.advice
follow_up	text	no	YYYY-MM-DD if sent at all	follow_up_date
notes	text	no	up to 2000	prescriptions.notes

An empty list and a missing field are not the same thing. "diagnoses": [] means "the doctor recorded none". Leaving the field out means "CMED did not send it", and that is reported as a problem. Please always send the list.

The order of items is kept exactly as you send it, and stored as the line number on the prescription.

14 · Corrections, repeats and retries

Situation	What to do	What happens
The doctor edits the prescription after it was sent	Send API 3 on Channel B again with the same five fields	We store it as a new version. Both are kept; the newest is the current one
Your network failed, you are not sure it arrived	Send it again, unchanged	Completely safe. You get 202 ALREADY_RECEIVED and one row, not two
You sent something wrong and want to replace it	Send the corrected version with the same five fields	Same as an edit
You sent a field we do not recognise	nothing	It is kept with the rest of the message. Unknown fields are never an error

Every message is stored word for word, exactly as you sent it, before anything is read out of it. So nothing is ever lost because we failed to understand a field. If we later realise we should have been reading something, we can go back through the stored messages and get it.

15 · Every reply from the server, and what to do about it

Code	Means	What CMED should do
202 ACCEPTED	Stored safely	Nothing. Mark it sent
202 ALREADY_RECEIVED	This exact message was already stored	Nothing. Retrying is safe
400 MALFORMED_JSON	The body is not valid JSON	Fix and resend. Do not retry unchanged
400 MISSING_FIELD	One of the five fields is missing or malformed	Fix and resend
400 INVALID_IDENTIFIER	An id has characters outside A–Z a–z 0–9 _ -	Fix and resend
401 INVALID_KEY	The key is missing or wrong	Stop and contact AIMS LAB
413 TOO_LARGE	The body is over 1 MB	Usually a mistake — check for a file or image in the message
422 SCHEMA_INVALID	Stored in quarantine , with a list of which fields were wrong	Not lost. Read the field list in the reply, fix, resend
500 / 503	Our server has a problem	Retry with a growing delay. Never drop the message

Please queue and retry rather than sending once and hoping. A message that is never sent leaves a recording that nobody can describe — and an unconfirmed recording is erased after 24 hours.

Shared — storage, testing and handover

For both developers, and for the project leads on each side

16 · Where every field ends up

So CMED can see that nothing is thrown away. All of this lives in a database called `aims_clinical`, separate from the one that tracks recordings — which deliberately holds no patient names at all.

You send	We store it in	Holding
the whole message, untouched	<code>intake_records</code>	exactly what you sent, once per message
demographics	<code>patients</code> , <code>encounter_demographics</code>	name, sex, date of birth, phone, address, age at this visit
paramedic	<code>paramedic_observations</code>	each reading in its own column, blood pressure also split into upper and lower, anything unrecognised in other
the visit itself	<code>encounters</code>	one row per consultation, linked to its recording
previous_visit	<code>encounters</code> (marked as a previous visit)	a second row, dated to that earlier day
items	<code>prescription_items</code>	one row per medicine , in the order you sent them
the prescription itself	<code>prescriptions</code>	with a version number, so corrections do not overwrite
diagnoses	<code>diagnoses</code>	one row each
investigations	<code>investigations</code>	one row each
female_details / male_details	their own tables	stored as sent

A copy of all of it is also written as a **JSON file beside the audio recording**, with the same name. So each consultation is one folder containing the audio and its clinical record together, and the database is the index into them.

17 · Rules that are easy to miss — both channels

The five fields, and the pairing

	Rule	Why
1	The five fields must be identical in all four messages	They are the only link between the recording and the clinical record
2	<code>start_time</code> in API 3 on Channel A and 3b is the original, from when the patient was opened	It is how the prescription finds its recording
3	Every timestamp carries its offset (+06:00)	A time without a zone is stored as empty
4	<code>hospital_id</code> must be the code AIMS LAB gives you	A recording claiming a clinic the laptop is not enrolled to is refused

The browser side

	Rule	Why
--	------	-----

5	Keep the session_id that API 1 returns	API 3 on Channel A needs it. Without it you cannot arm the gate
6	Treat RECORDING_PROVISIONAL as success	The microphone is already live. It is not a warning
7	But keep listening — a provisional start can be refused a moment later	The recording is then stopped and deleted, and the page must stop saying "recording"
8	prescription_built does not stop anything	The counselling after it is valuable audio
9	Act on code, never on message	The wording may change; the code will not
10	Reconnect if the connection drops , and reconnect quietly	A dropped socket does not stop a recording in progress
11	Give AIMS LAB every web address you will use	Staging and preview addresses are refused unless listed
12	Never send stop, pause or resume	Those belong to the doctor
13	If the connection will not open at all, say "AIMScribe is not running"	It usually means the program is not started on that PC

The server side

	Rule	Why
14	Never let a message delay the doctor. Send from a background queue	A slow network must never make Build Prescription feel slow
15	previous_visit must be present , even if null	Missing means "CMED forgot"; null means "nothing to send"
16	items, diagnoses and investigations are always lists , empty if none	Same reason
17	sex is the word female or male	Codes like M, F, 1, 2 are rejected
18	The key never reaches a browser	It is a server credential
19	Retry on 500 and 503 , with a growing delay	Retrying is safe; dropping the message is not
20	Unknown fields are welcome	They are kept. Do not strip data to fit

18 · A complete working example

The browser half of the integration, with the five fields built once and reused by all four messages. Everything else in this document is detail.

```
let socket, sessionId = null;

function connect() {
  socket = new WebSocket("ws://127.0.0.1:5050/ws");

  socket.onmessage = (e) => {
    const m = JSON.parse(e.data);

    // Unprompted status - the reliable way to show what is really happening.
    if (m.event === "status") { showRecordingState(m); return; }

    if (m.command === "start") {
      if (m.code === "RECORDING_STARTED" || m.code === "RECORDING_PROVISIONAL") {
        sessionId = m.data.session_id; // keep it - API 3 on Channel A needs it
        showRecording(true);
      } else {
        showProblem(m.code); // act on the code, not the message
      }
    }
  };

  socket.onclose = () => { showRecording(false); setTimeout(connect, 2000); };
  socket.onerror = () => showProblem("AIMScribe is not running on this PC");
}

// The doctor opens a patient.
// Build the five fields ONCE, here, and reuse this object everywhere.
function openPatient(patient, doctor) {
  const visit = {
    patient_id: patient.id,
    doctor_id: doctor.id,
    hospital_id: CLINIC_CODE, // the code AIMS LAB gave you
    start_time: new Date().toISOString(), // with offset
    date: localDateString() // the clinic's local day
  };
  currentVisit = visit; // keep for API 3 on Channel A and 3b

  socket.send(JSON.stringify({
    command: "start",
    request_id: "open-" + visit.patient_id,
    trigger: visit // the SAME object
  }));

  sendPatientInformationFromYourServer(visit, demographics, paramedic); // API 2
}

// The doctor presses Build Prescription.
function prescriptionBuilt(prescription) {
  socket.send(JSON.stringify({
    command: "prescription_built",
    request_id: "built-" + currentVisit.patient_id,
    patient_id: currentVisit.patient_id,
    session_id: sessionId
  }));

  // API 3 on Channel B - note it reuses currentVisit, so start_time is the ORIGINAL.
  sendPrescriptionFromYourServer(currentVisit, prescription);
}

connect();
```

19 · How CMED can test, today

You do not need to wait for us, and you do not need a real patient.

	What	How
1	Check a server message against the rules, with nothing running	<code>python tools/channel_b_test.py --local</code>
2	Send real server messages to a test server	<code>python tools/channel_b_test.py --server https://<host> --key <your key></code>
3	See every message and reply side by side	open /protocol-test on the test site
4	Test the browser signals against a real recorder	install the recorder on one PC, then open your page and watch the replies
5	Confirm the server is up at any time	<code>curl https://<host>/health</code> — no key needed

Step 2 runs every case that matters — a good message, the same message twice, a broken one, a changed prescription, a wrong key, an oversized body — and checks each reply against this document. **Please run it and send us the output before the first clinic goes live.**

20 · What each side must hand over

AIMS LAB gives CMED	CMED gives AIMS LAB
The server address	The exact web address of your page (for example <code>https://ehr.aaloclinic.com</code>) — including staging
The Channel B key — shown once	Your own clinic codes , so we can map them to ours
The seven clinic codes	A test environment we can send to
This document	A named contact for integration

The two we need from CMED are blocking. Without your page's exact address the browser cannot talk to the recorder at all — nothing works. Without your clinic codes, a visit you describe cannot be matched to the clinic it happened in: the record is stored, **you get a normal success reply**, and the **recording is quietly erased after 24 hours**. That failure is completely silent from your side, which is why it matters more than it sounds.

21 · One page for the CMED developer

Question	Answer
How many messages do I build?	Four. Two from the browser (Part B), two from your server (Part C)
What ties them together?	The five fields, identical in all four
Biggest trap	start_time in the prescription messages is the original , not the time the prescription was built
Browser — where do I connect?	<code>ws://127.0.0.1:5050/ws</code> — the same PC, every time
Do I need a password there?	No. You need your page's web address on our allow-list instead
What do I keep from the reply?	session_id — API 3 on Channel A needs it
Is RECORDING_PROVISIONAL bad?	No. It means recording. But keep listening — it can still be refused
Does prescription_built stop it?	No. It arms the gate. The next patient closes the consultation
What stops a recording then?	The next API 1, or the doctor's own Stop button
Can I send stop or pause?	No. Those belong to the doctor
What if the socket drops?	Reconnect quietly. The recording continues regardless

What if it will not connect?	Show "AIMScribe is not running". The program is probably not started on that PC
Server — where do I send?	/api/v2/clinical/patient-information and /api/v2/clinical/prescription
How do I authenticate?	Header X-CMED-Key. Server to server only, never in a browser
What is required in API 2?	demographics, and previous_visit present (null is fine)
What is required in API 3 on Channel B?	issued_at, items, diagnoses, investigations — lists may be empty
What if a reading is out of range?	Stored empty. It never rejects the message
What if I send extra fields?	They are kept. Never an error
Is retrying safe?	Yes. Same body twice gives one row and ALREADY_RECEIVED
What if I get 422?	Nothing is lost — it is quarantined. Read the field list, fix, resend
When do I send previous_visit?	Whenever we might not have it. For the first release, send it every time
Can a message slow the doctor down?	It must not. Queue it and retry in the background
How do I test?	tools/channel_b_test.py, and the /protocol-test page

Prepared by AIMS LAB, United International University. This document replaces the two earlier separate specifications and carries all of their content. Every rule here is taken from the code that handles these messages. If anything is unclear, ask before building — it is far cheaper than discovering it in a clinic.